

Security in Software Applications

Smart Contracts - Crash Course



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

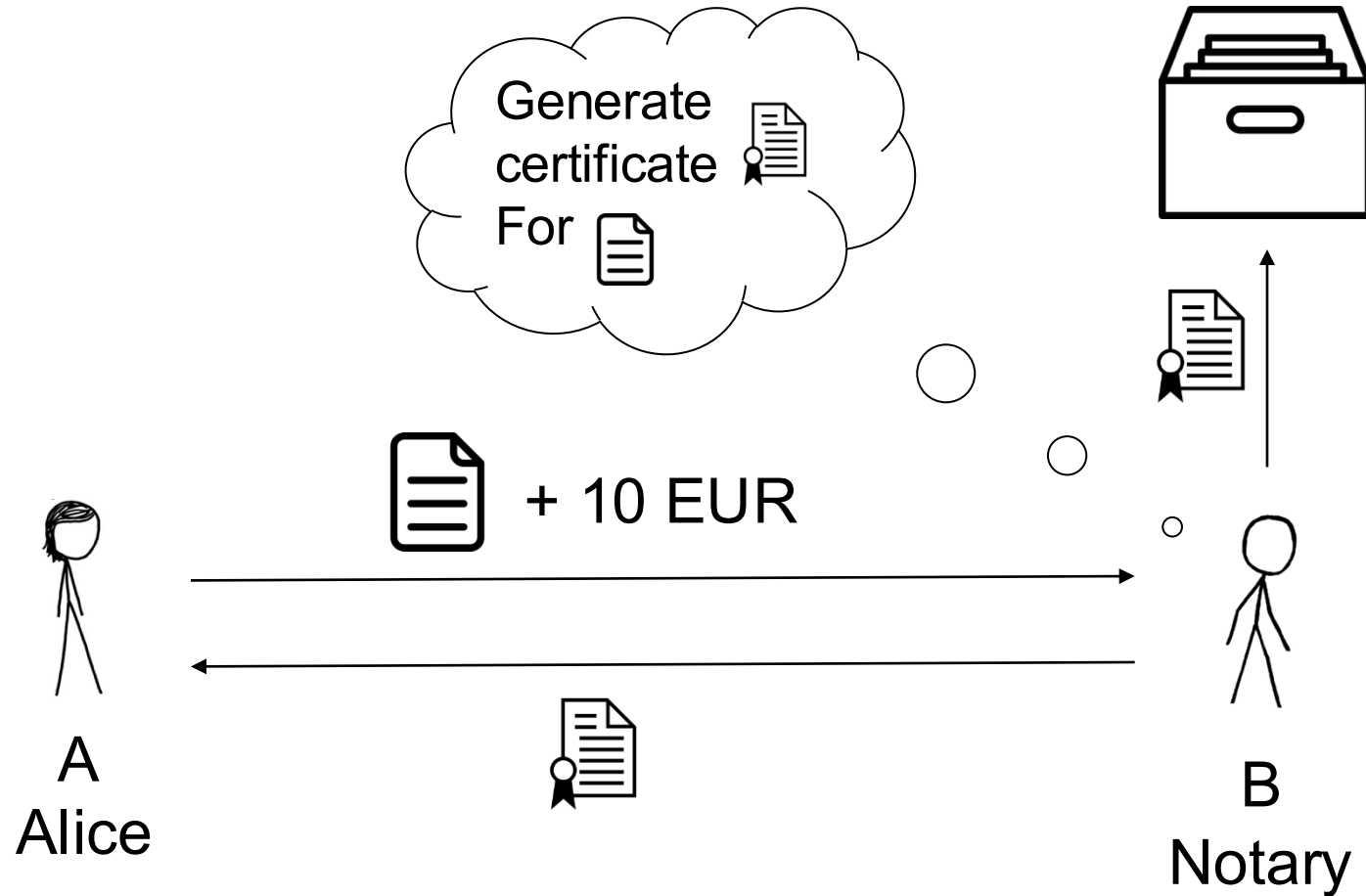
The Notarization Problem

- Alice wants to buy a house.
- With the aid of some entity (e.g. real estate company) produces a contract
- The contract C must be *notarized* and *deposited* somewhere.
- What do we need to conclude the procedure?
 - A **notary**, whose aim is to validate the contract
 - A secure **archive** where the contract can be deposited

The **notary** must be trusted: he must sign *only* correct contracts

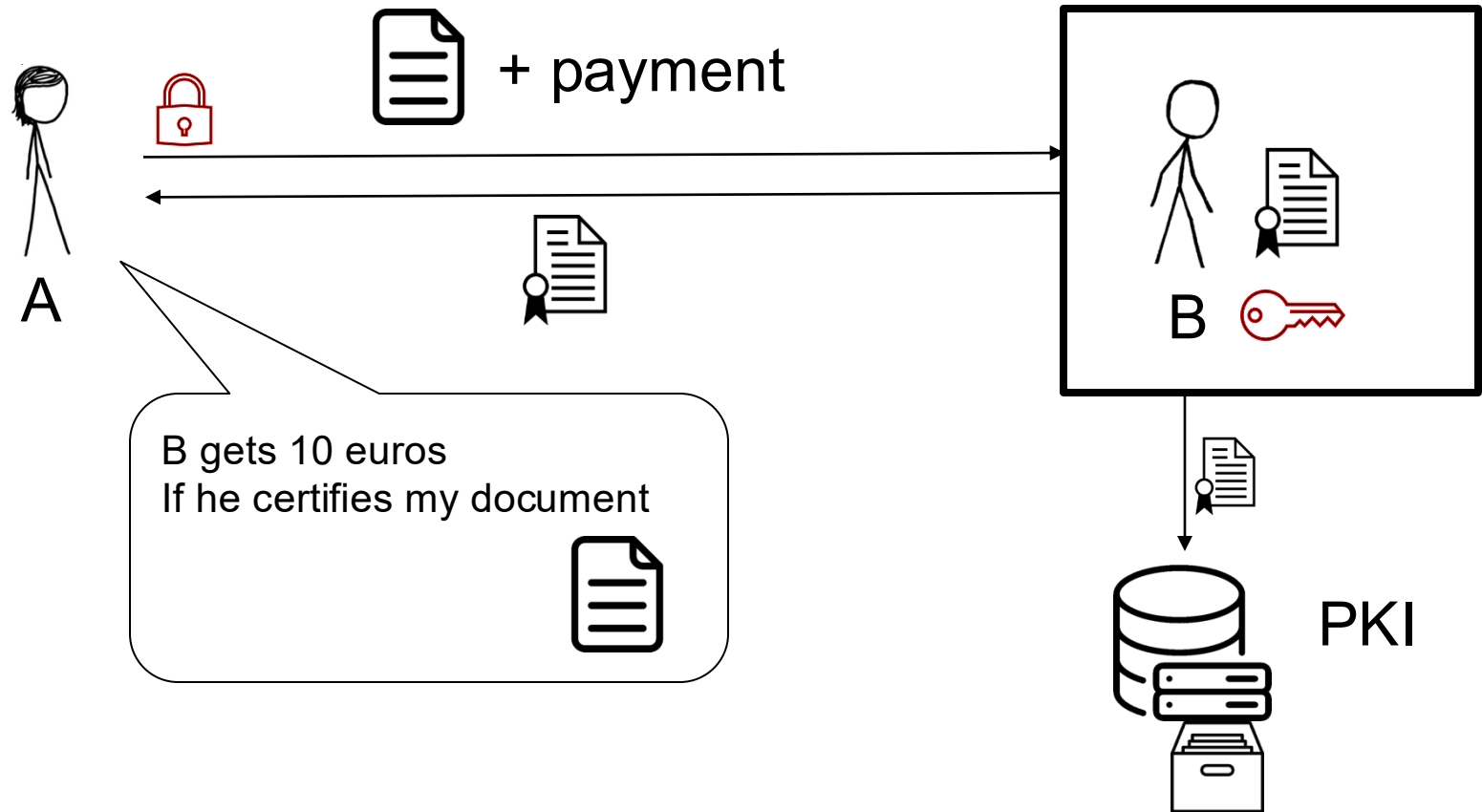
The **archive** must be *publicly* accessible and should be *untamperable*

Notarization example – Basic interaction



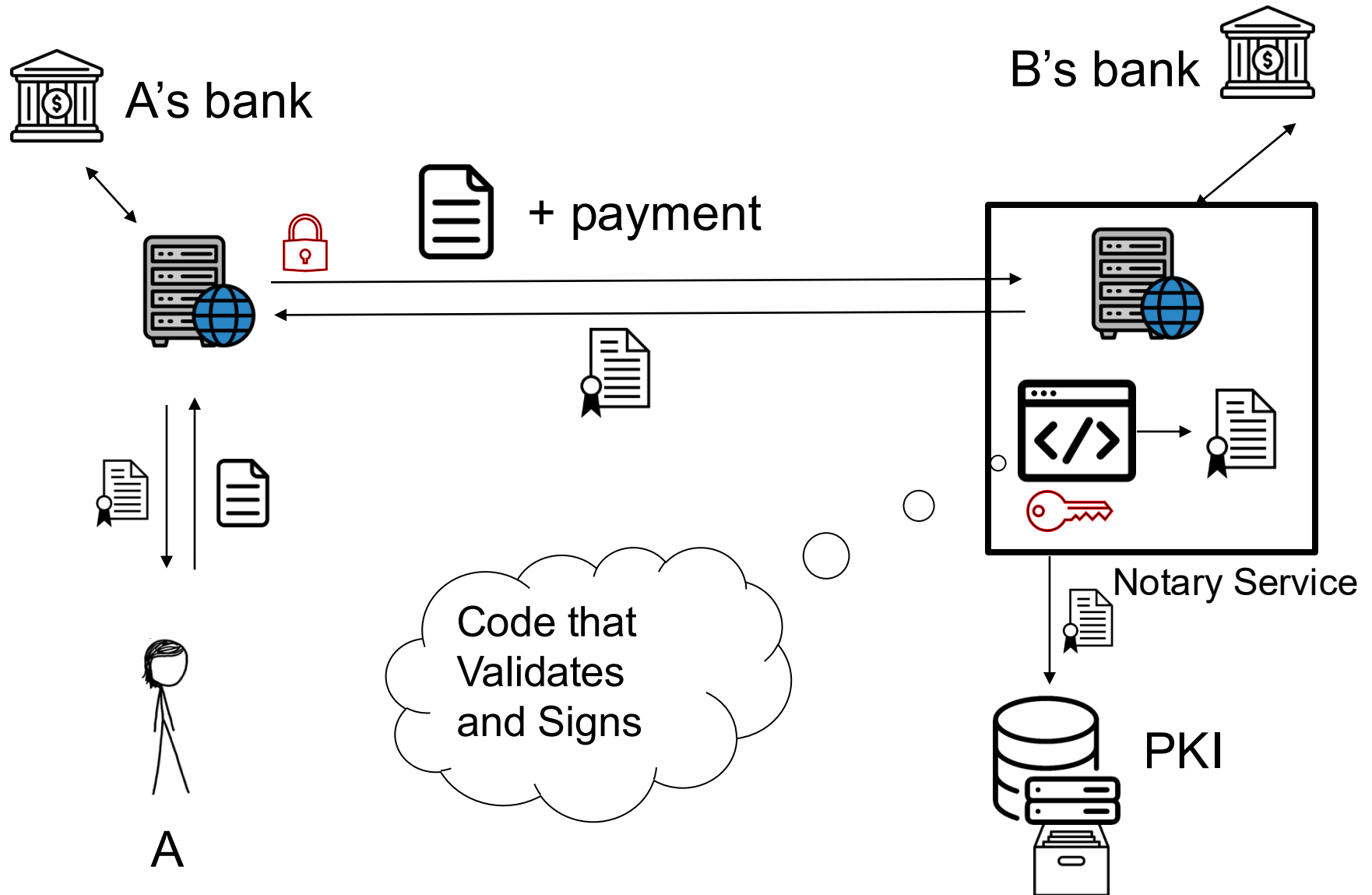
The certificate is a document signed by the notary

Digital Interaction with the notary



 = Secure connection (e.g. HTTPS)

Automatic Notarization – First attempt



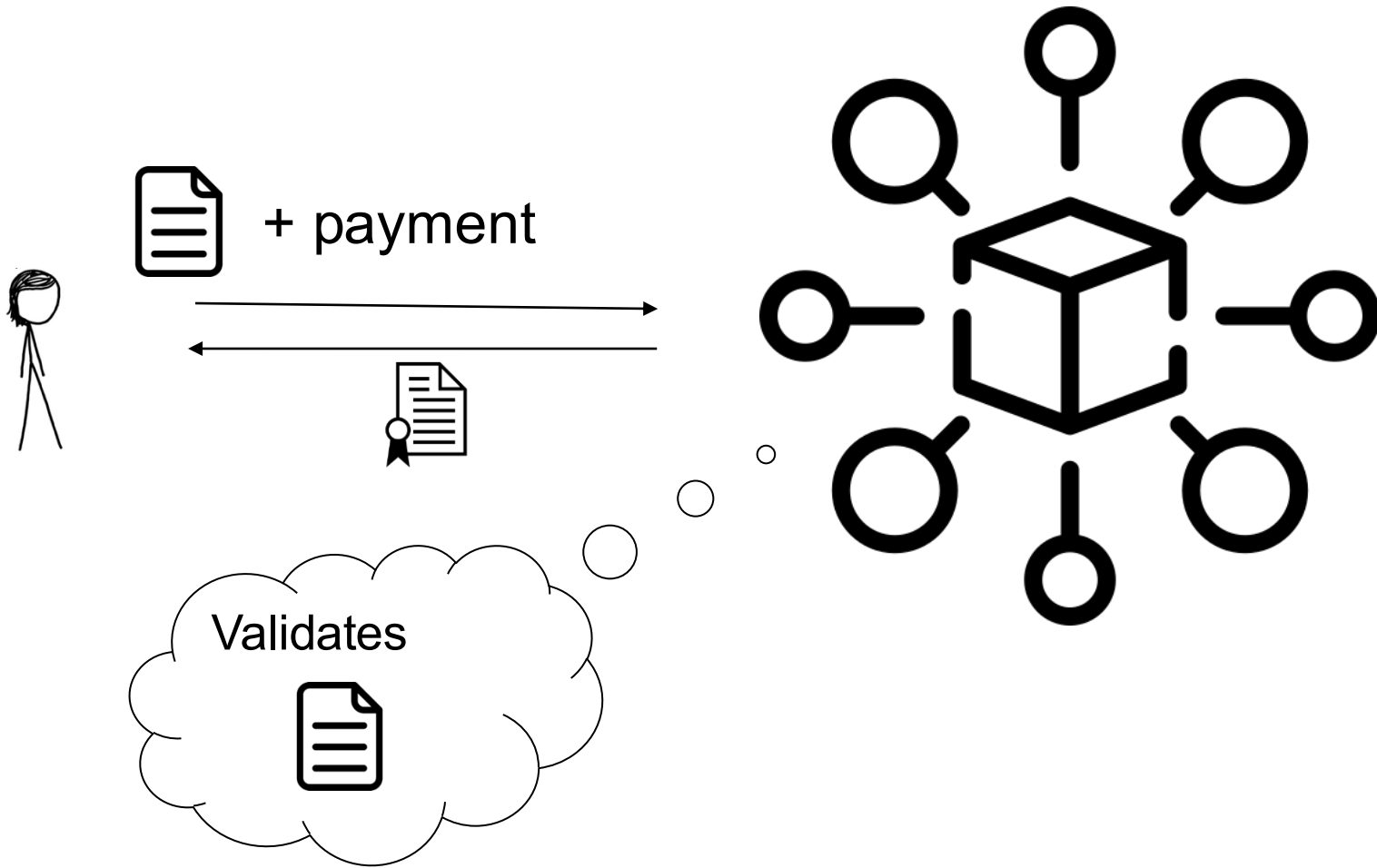
Automatic Notarization issues

- Other than common software vulnerabilities issues in:
 - Server's code
 - Notarization program code

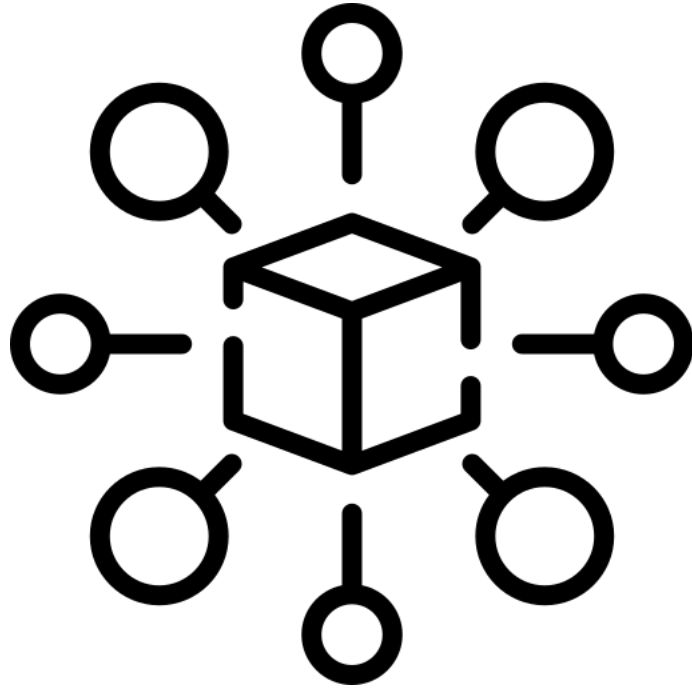
We have that an untrusted server might be compromised!

- ➔ Integrity issues: Corrupted notary server will notarize invalid contracts
- ➔ Availability issues: Compromised database might be violated, and records *deleted*, DoS attacks
- ➔ *Extra*: we need to trust banks and banks' servers

Second attempt - Decentralize everything!



Second attempt - Decentralize everything!



This system should be

- Programmable
- Publicly accessible
- Publicly verifiable
- Immutable
- Manages money!

→ Solution: Blockchains!

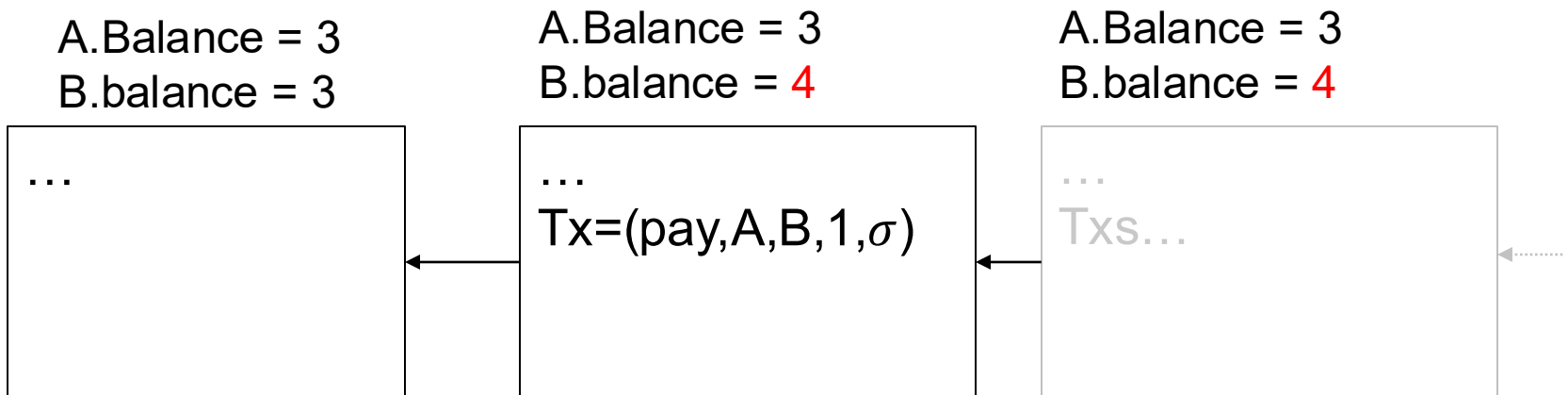
→ introduced by for the first time by Satoshi Nakamoto in 2008 had revolutionized the idea of currencies (decentralized currency)

→ <https://bitcoin.org/bitcoin.pdf>

→ First notion of programmability (scripts)

What is a blockchain?

- A **blockchain** is a decentralized list of blocks that can be extended.
- In the Ethereum blockchain, accounts are represented by signature public keys (addresses), used to verify transactions.
- Account Balances can be recomputed by looking at the transactions inside the blocks
- A block contains:
 - Set of *valid* signed transactions justifying balance updates
 - Other informations needed to verify block *validity* (e.g., hash the previous block, ...)



Tx is included in a block if

- the signature σ verifies under A's public key
- A has enough balance

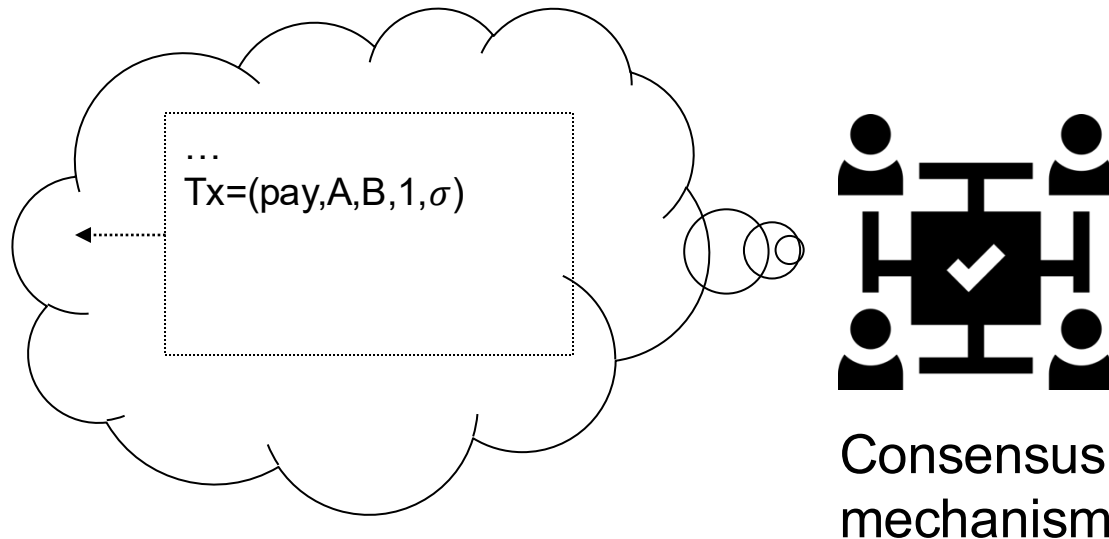
Blockchain properties

- **Persistence:** Blocks are always available and cannot ever be modified.
- **Liveness:** If a valid transaction is issued to the blockchain, it will eventually appear in a block.
- **Public Verifiability:** Anyone can download and verify the validity of the blockchain.

Such properties are guaranteed if less than $1/3$ (or $1/2$ depending on the consensus) of nodes is corrupted.

A good blockchain has **thousands** of nodes keeping it alive.

Who proposes new blocks? Where such currency comes from?



Block proposal and currency minting: Proof-of-work (BTC)

- Who mints currency?
 - As for gold, smth physical must correspond to smth «logical»
 - Consensus nodes can verify that some work has been done to mint and assign some currency
 - **Coinbase transaction:** *special transaction assigning coins to the winning proposer based on the proof that some computational work has been done.*
 - **HOW??**

SHA-256 hash function outputs are random-like strings

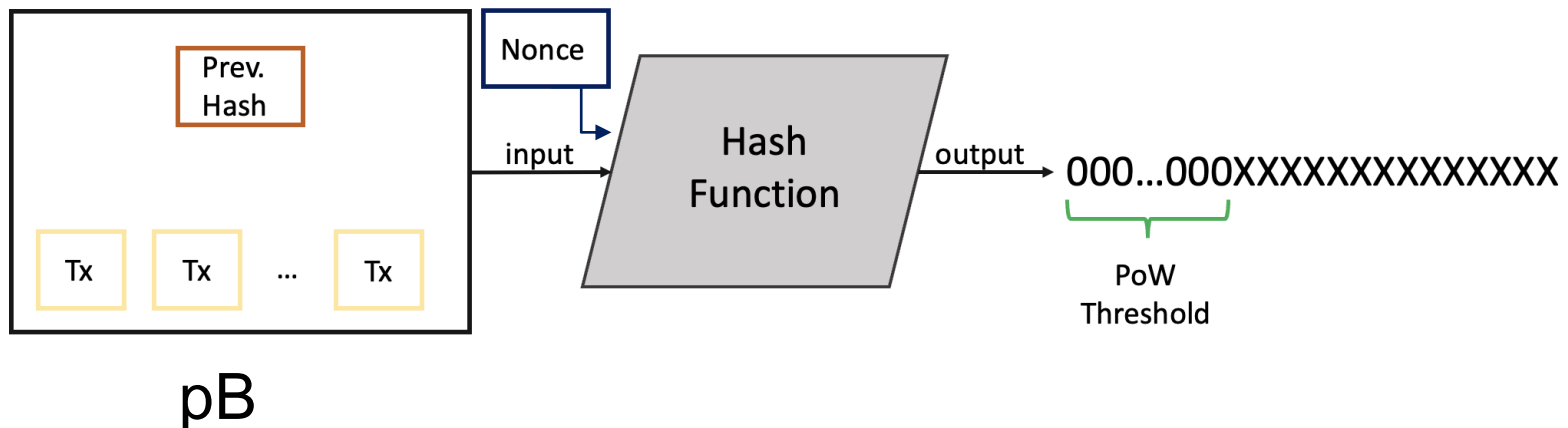
$\text{Sha256}(\text{ab00}) = 3a1eef43a87fff51d535b1025e31d8c8fed3109eec247ca6beb252333d2f4fac$
 $\text{Sha256}(\text{ab01}) = a61df51f631f44da66aeaeb67ccec39d04812efe5c8246aacb70c780198bb2b$

Given $3a1eef43a87fff51d535b1025e31d8c8fed3109eec247ca6beb252333d2f4fac$

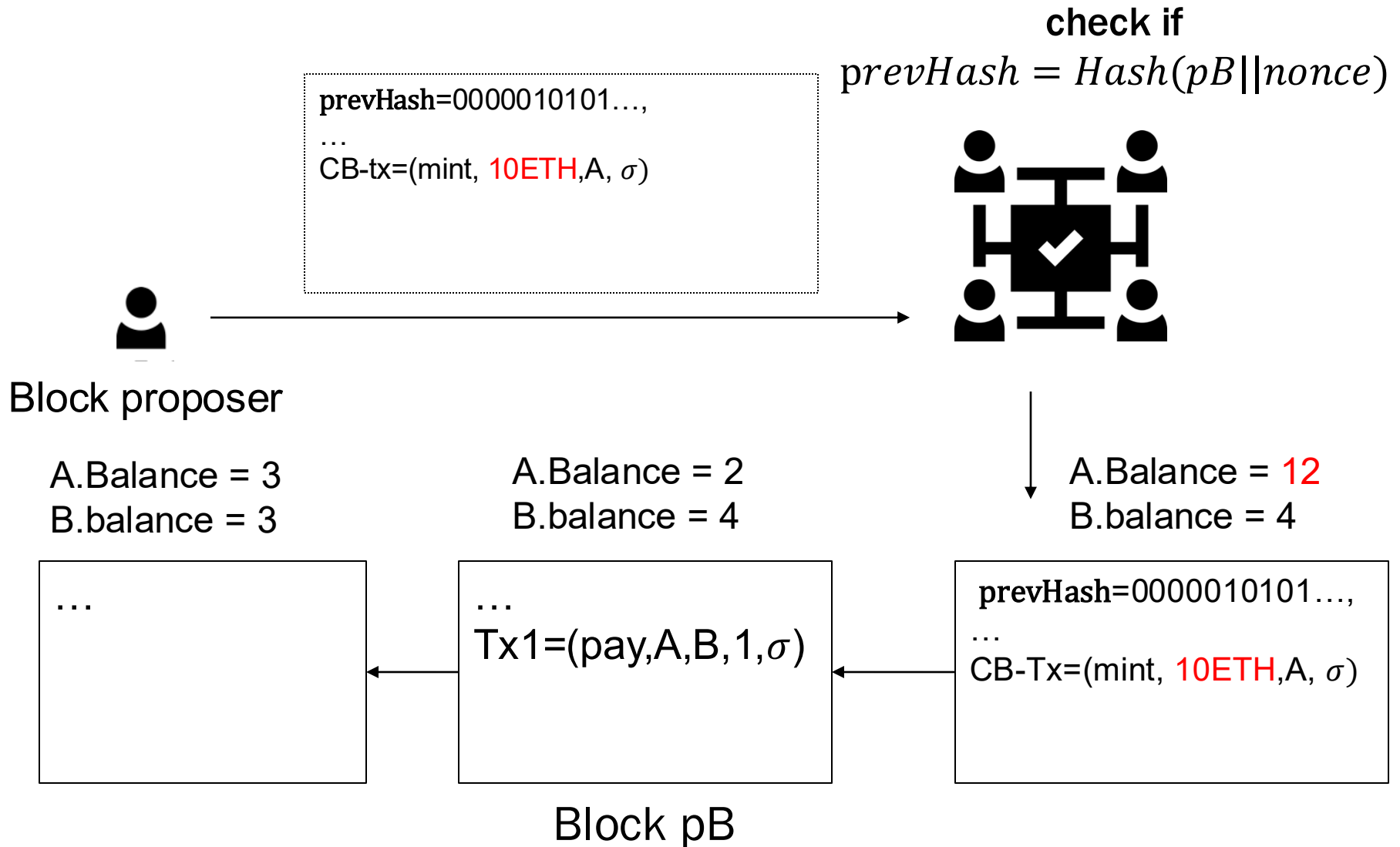
- The input `ab00` can be guessed only with brute force
- Hash outputs are unrelated

Proof-of-work

- Brute forcing on long inputs needs a lot of computational power
- A node, to earn currency, must propose a new block B incorporating the hash of the previous block pB concatenated with a *nonce*.
 - $\text{Hash}(pB || \text{nonce}) = H$
 - The new block is valid if $H < T$, for a threshold T
- *In other words, $\text{Hash}(pB || \text{nonce})$ shall contain at least a certain number of 0s at the beginning.*
- After finding $(x=(pB, \text{nonce}), \text{Hash}(x)=0000\dots)$, the proposed block contains the hash pair $(x, \text{Hash}(x))$, the txs (taken from a pool of received transactions) and *the coinbase transaction*. Consensus nodes, upon block validation, will append the new block to the chain.



Proof-of-work



Ethereum Smart Contracts – Programmable Money

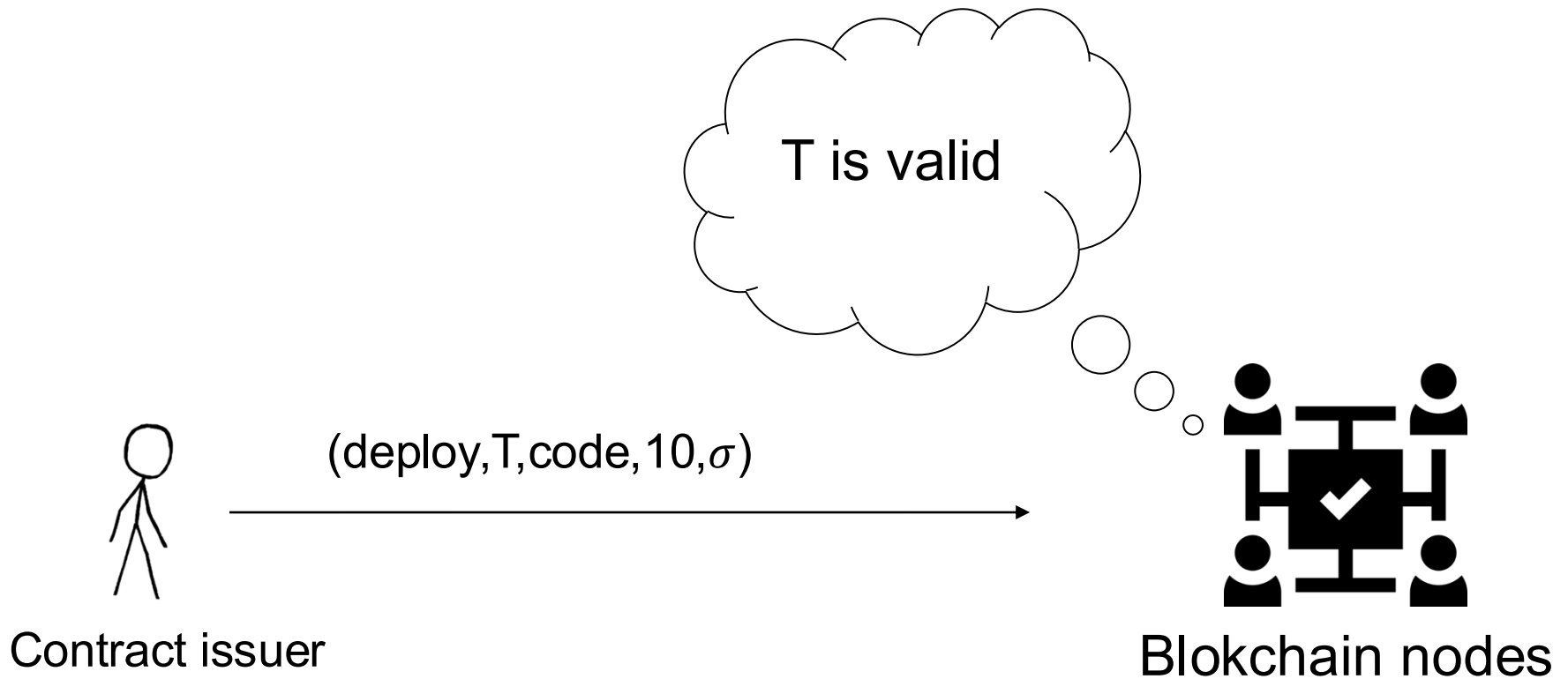


- Any user with an account can deploy a turing-complete program (*contract*) through a blockchain transaction.
 - The *Contract C* is an account of his own (it has an address)
 - Users interact with the contract by sending contract transactions, including currency
 - Blockchain and contract states are calculated by the nodes within the *Ethereum Virtual Machine*
- Smart Contract functions provide an interface to
 - exchange ETH under certain conditions
 - Manage an *immutable database*
- They enable a lot of interesting applications:
 - *Alternative currencies: **Alternative tokens*** with some ETH value.
 - Decentralized Finance (DeFi)
 - Decentralized Autonomous Organizations (DAO)
 - *NFT*: A unique ID assigned to the token guarantees its non-fungibility.
 - Include data such as images: Decentralized Art.
 - Decentralized Notaries
 - ...

Basic Token Smart Contract

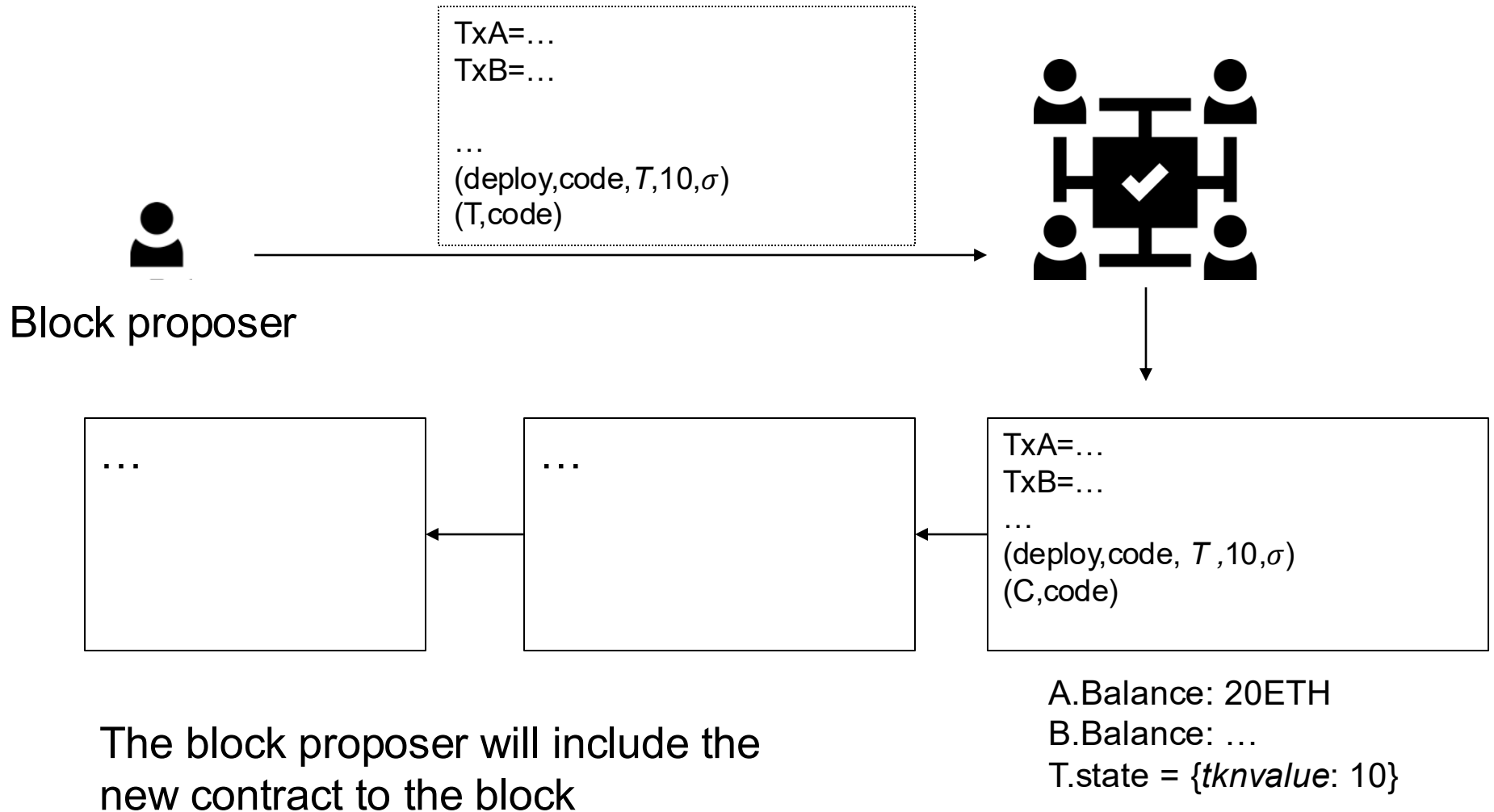
```
Contract Token {  
    int tknvalue;  
    map balance (address=>int)  
  
    ...  
  
    void deploy(int v) { tkn = v; }  
  
    //Function to exchange ETH with Tokens  
  
    void exchange (int quantity) payable {  
        //sender.sent internal variable outputting the number of ETH sent together  
        with the function input  
  
        If (sender.sent >= quantity*tknvalue) balance[sender] += quantity*tknvalue  
    }  
  
    //Auxiliary function to retrieve the current user balance  
  
    int getBalance() { return balance[sender]; }  
}
```

Contract deployment

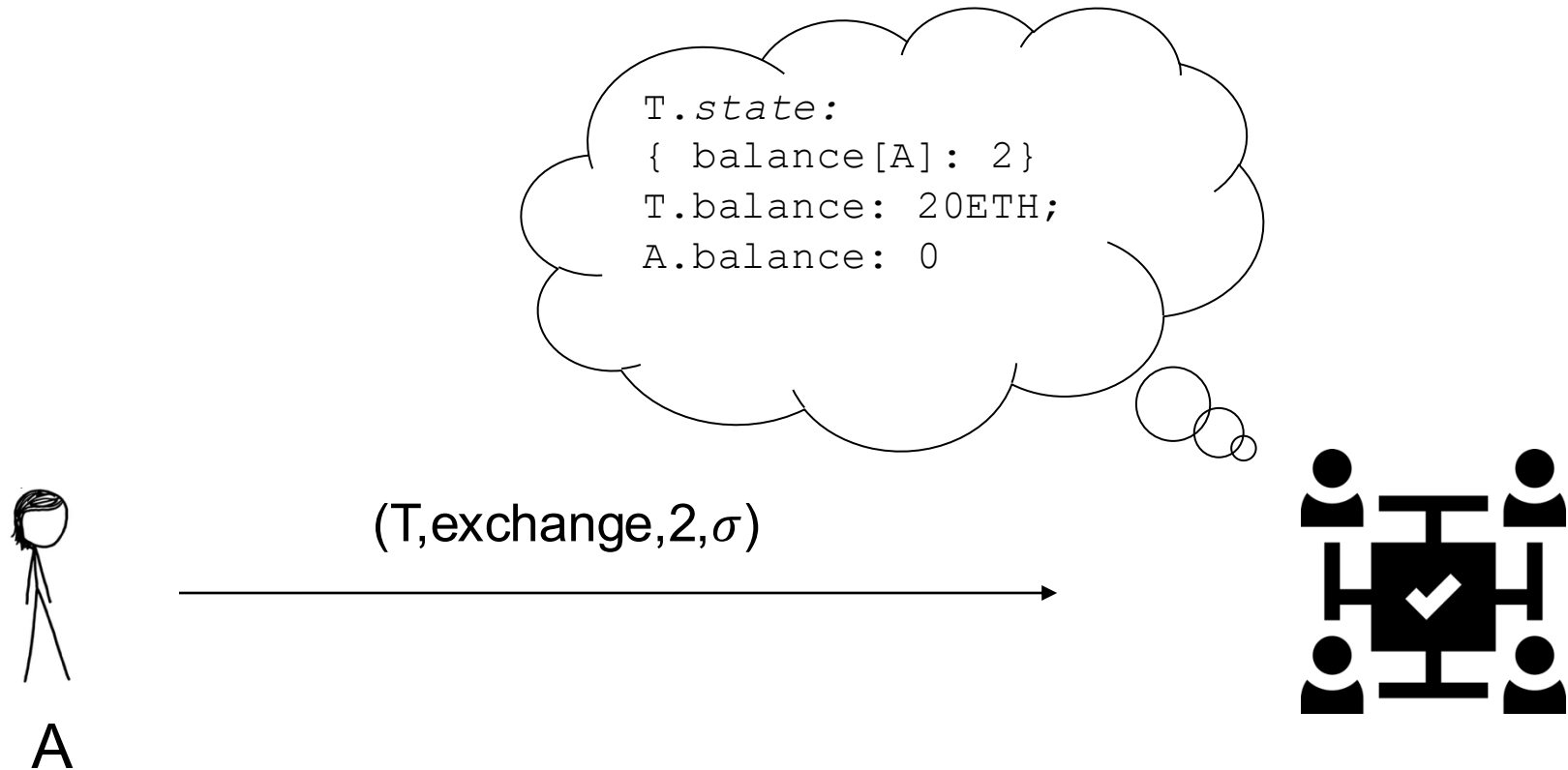


- Bob deploys a new contract to the Blockchain through containing:
 - The code
 - The value of the contract (currency)
 - The issuer signature

Contract execution

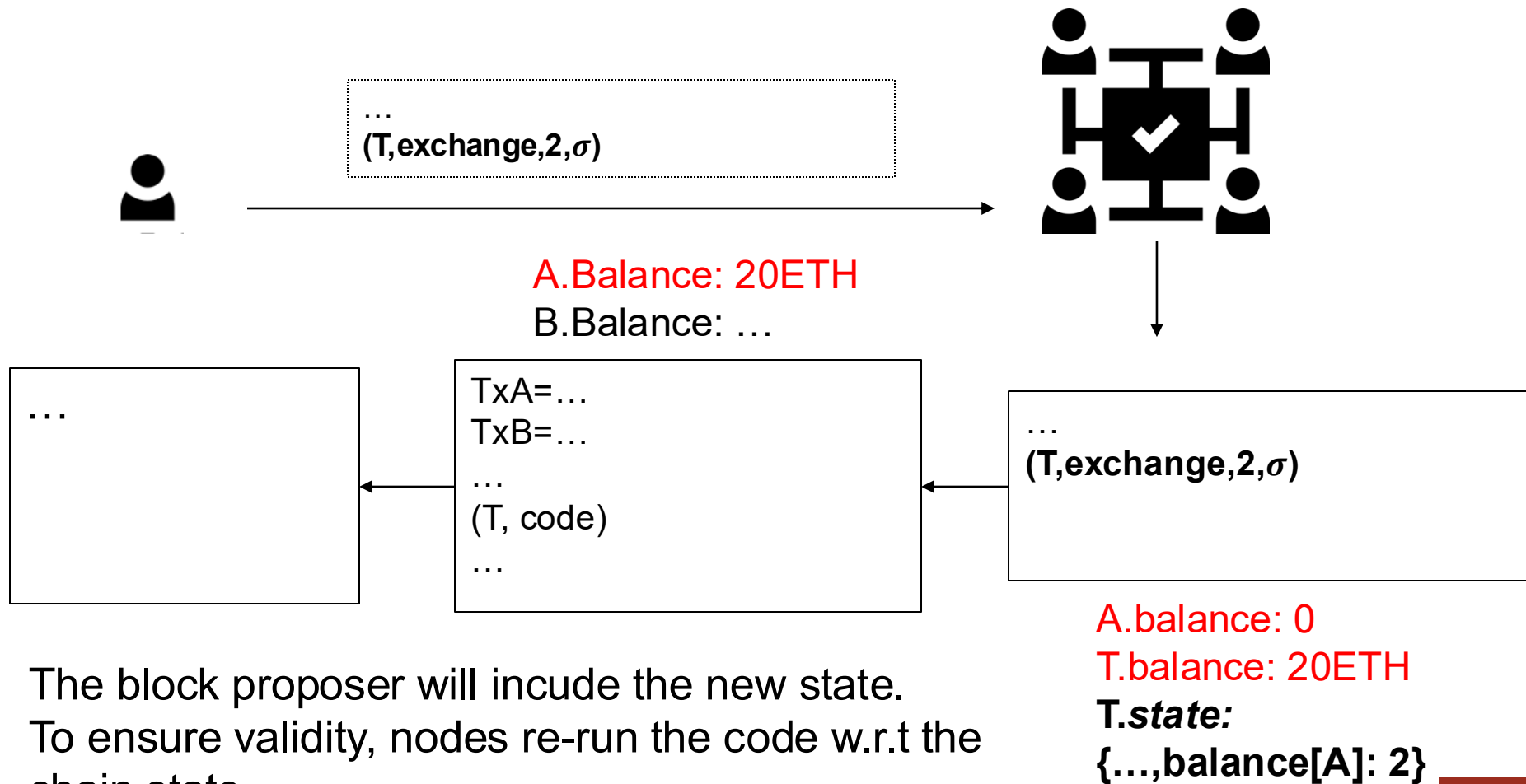


Contract execution



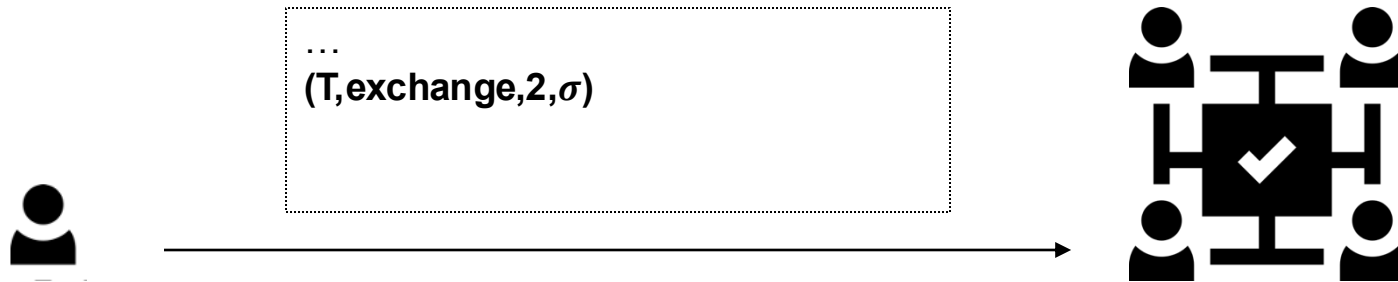
All the nodes will run `exchange (2)` triggered by A to agree on the function output

Contract execution



Contract execution

REJECT



Nodes will reject the transaction if Alice does not have enough ETHs to exchange 2 tokens (10 ETHs)

Simple Notary Smart Contract

```
Contract Notary {

    \\ Variables containing parameters for notarization

    string document;
    boolean notarized;

    ...

    \\ deployment info, taking as inputs notarization parameters
    void deploy( ... ) {
        notarized=false; ... }

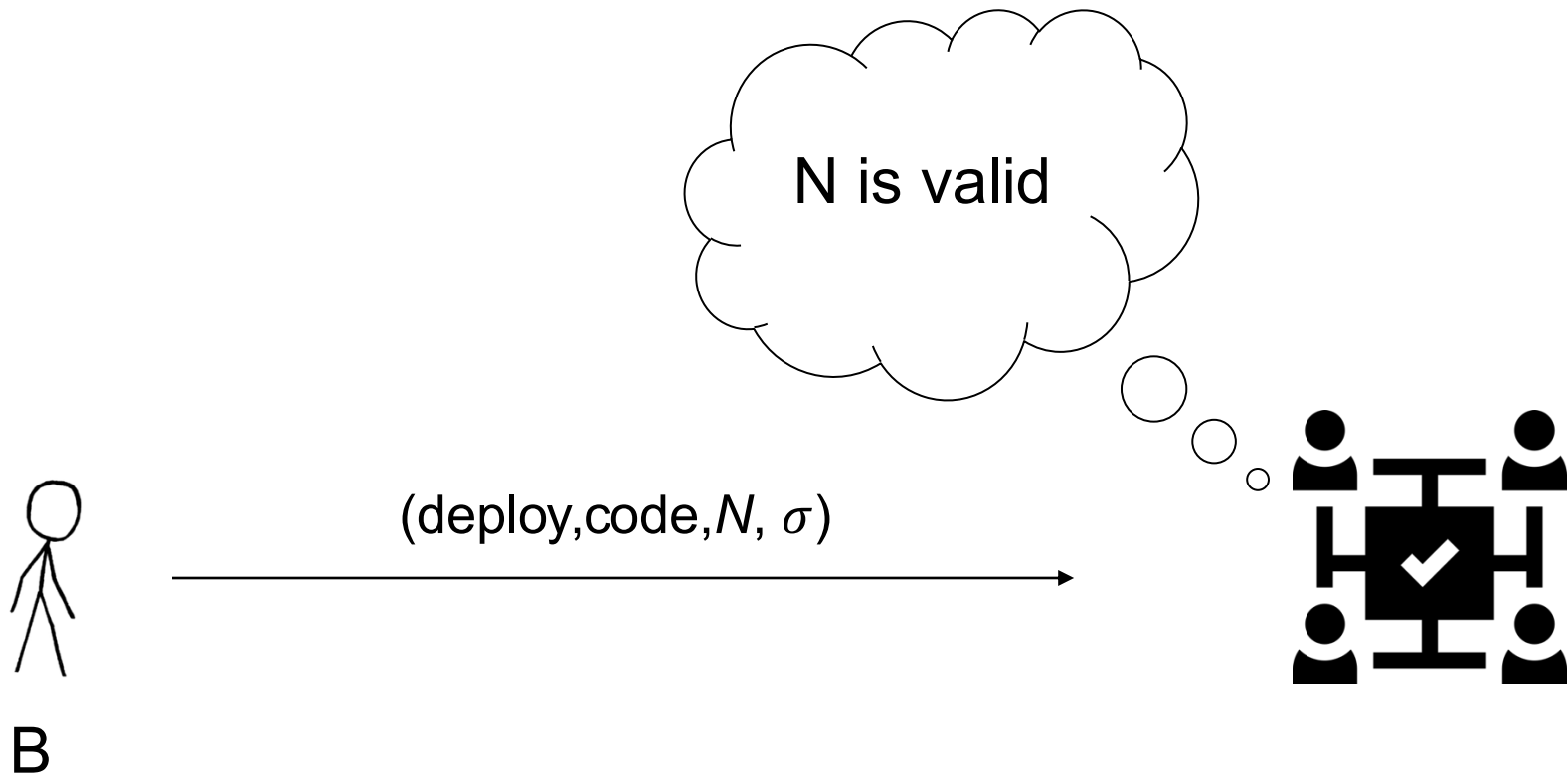
    \\Function to be triggered by the user
    void notarize ( string D, ... ) {
        if (checkValidity(document,...)) {
            document = D;
            notarized = true }}

    \\ Checks that wether contract can be notarized, i.e. document is well
    written, involved parties signed the document, etc.

    boolean checkValidity(string s, ...) { if (...) return true }

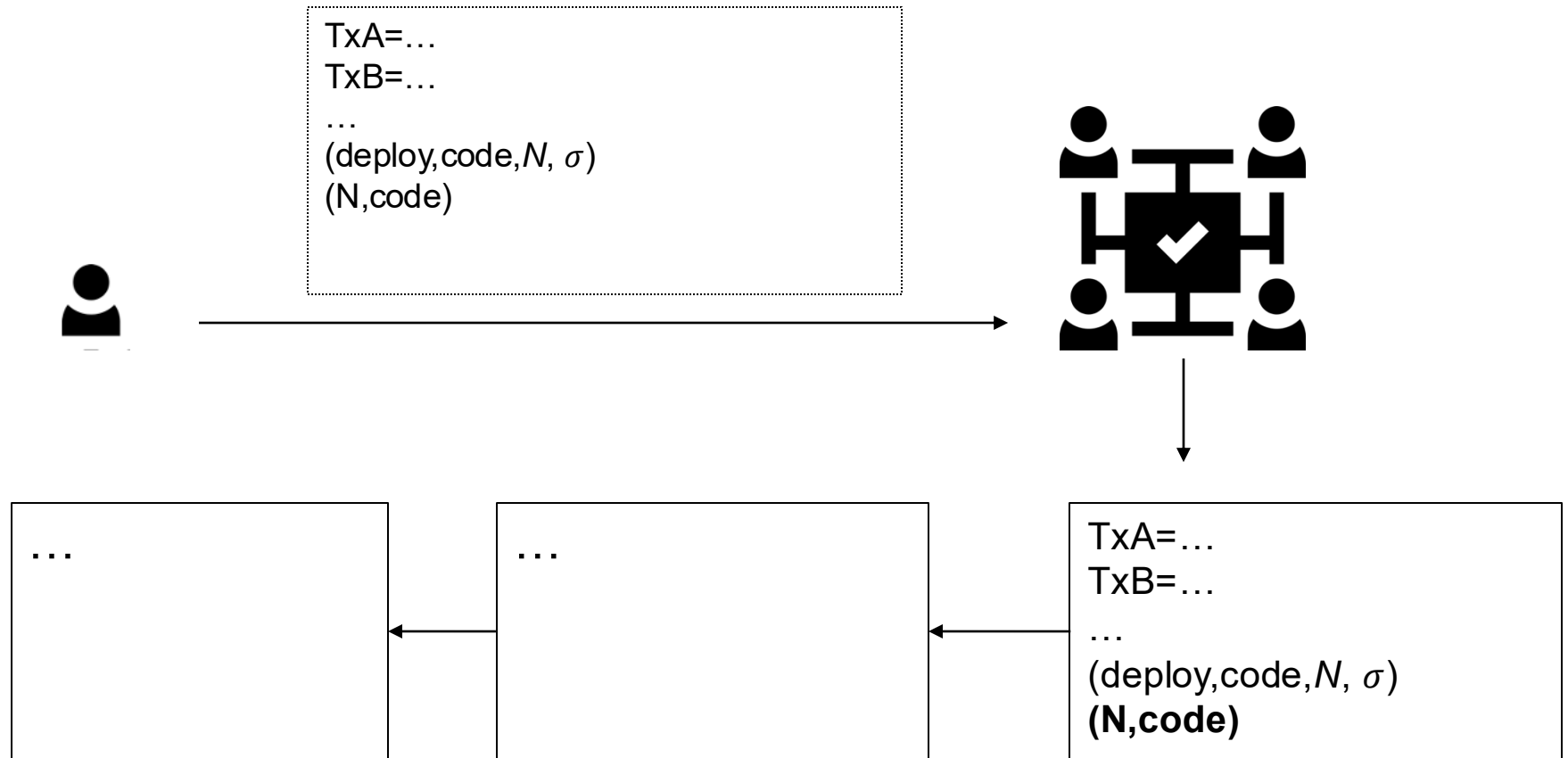
}
```

Contract deployment



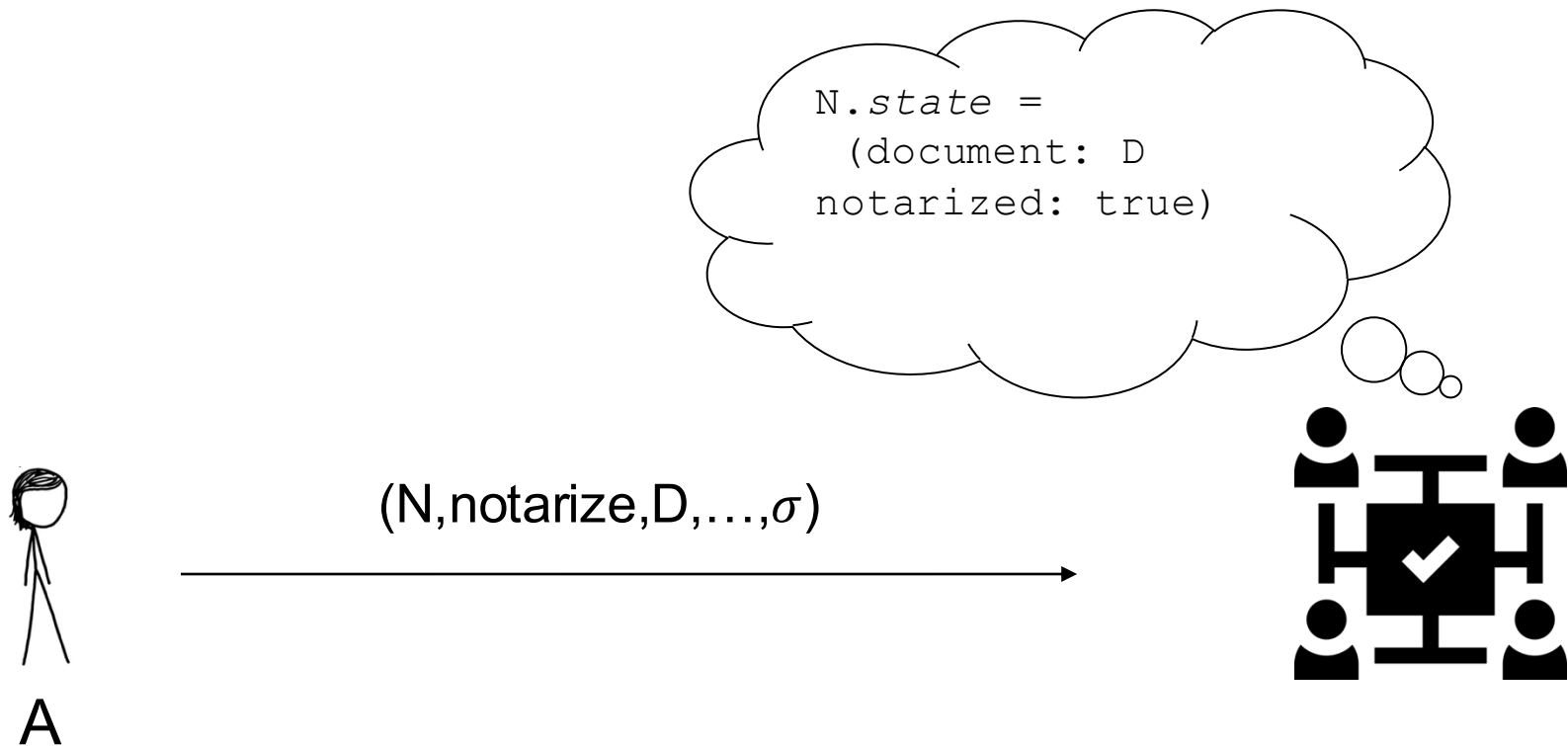
All the nodes will check the contract validity

Contract execution



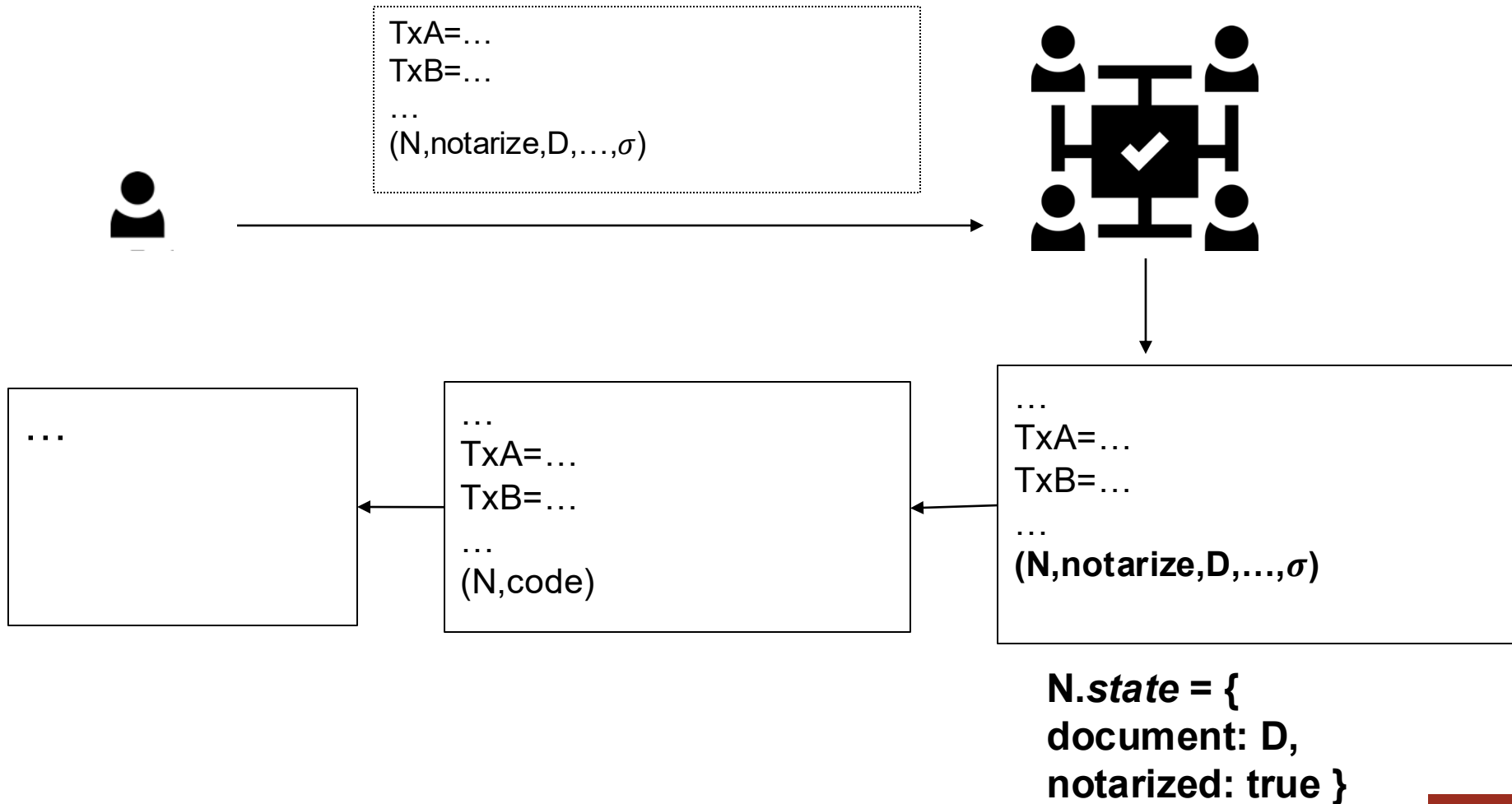
The block proposer will include the new contract

Contract execution

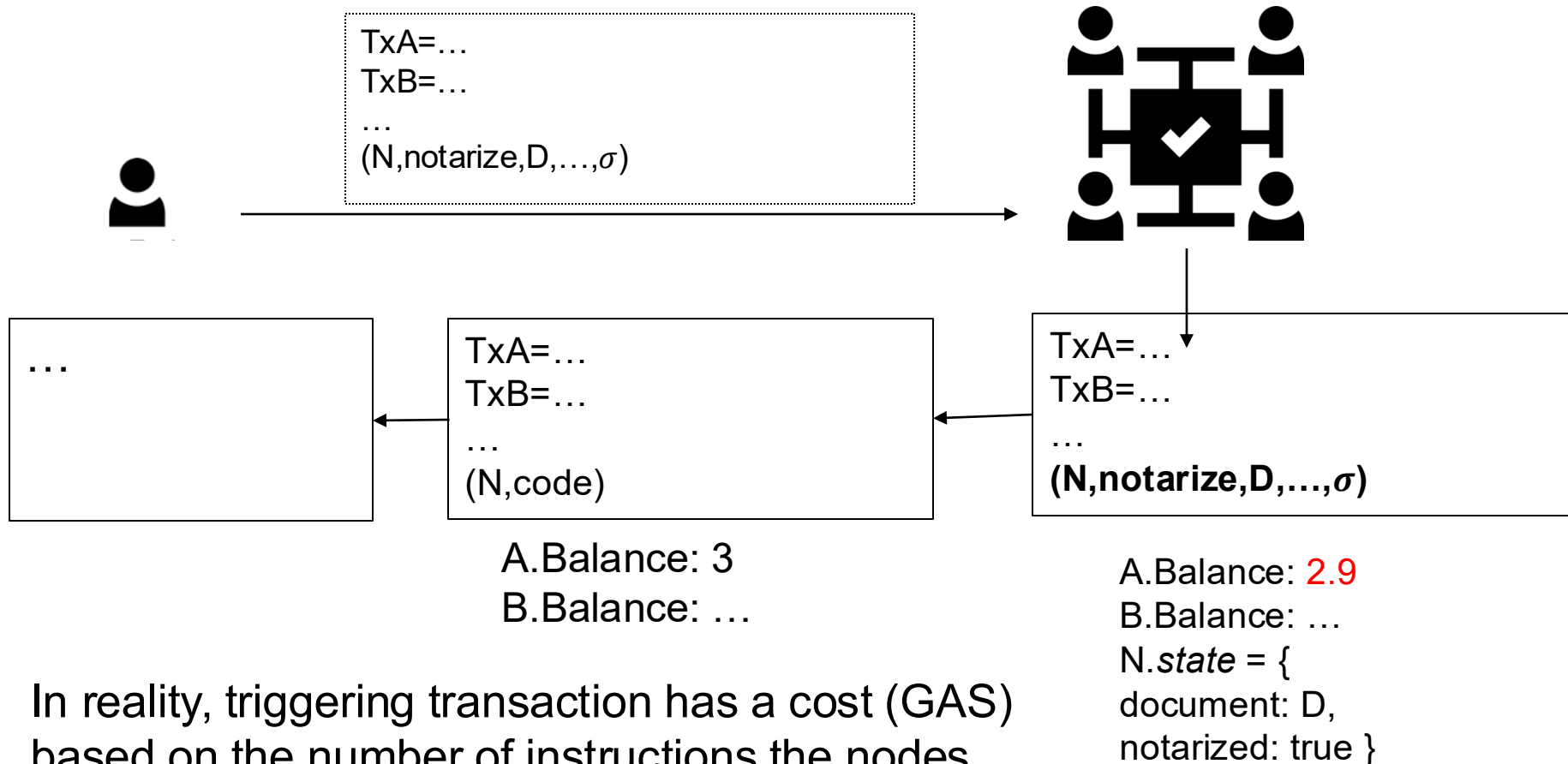


All the nodes will run `notarize(D, ...)` and agree on the new state of the contract `N`.

Contract execution



Contract execution: GAS



In reality, triggering transaction has a cost (GAS) based on the number of instructions the nodes have to run

Alternative Consensus Mechanism – Proof of Stake

Stake is an amount of crypto-commodities invested into the system

- Locked via a special transaction, or sent to a specific address
- Cannot be spent

Rationale

- The more is left at stake by users, the less likely they would want to subvert the blockchain
- Validators place their “bet” on the chain(s) they deem true
- If the block is appended, validators get a reward proportional to the bet

No block mining (puzzle solving) reward

- But also nothing to lose! (No expenses due to hardware and energy consumption)

Block proposers' election probability depends on their stake value

Takeaways

- Smart contracts are written by developers:
 - It can be prone to most of the known software vulnerability!
 - Software security is even more critical when dealing with currencies
 - New attacks related to the new contexts can come out
 - In practice, a smart contract is nothing more than a piece of code running on top of a Virtual Machine.
 - With the knowledge of the underlying architecture (e.g., Ethereum Blockchain) and the (E)VM environment, symbolic analysis and fuzz testers can be implemented as well by adapting the standard techniques to this new environment.
- Software vulnerabilities led to destructive attacks:
 - DAO Attack: \$150 million worth of ethers stolen due to a smart contract bug
 - By carefully exploiting smart contract callbacks, the attacker could change smart contract ownership to any account of his choice.
 - <https://www.coindesk.com/consensus-magazine/2023/05/09/coindesk-turns-10-how-the-dao-hack-changed-ethereum-and-crypto/>